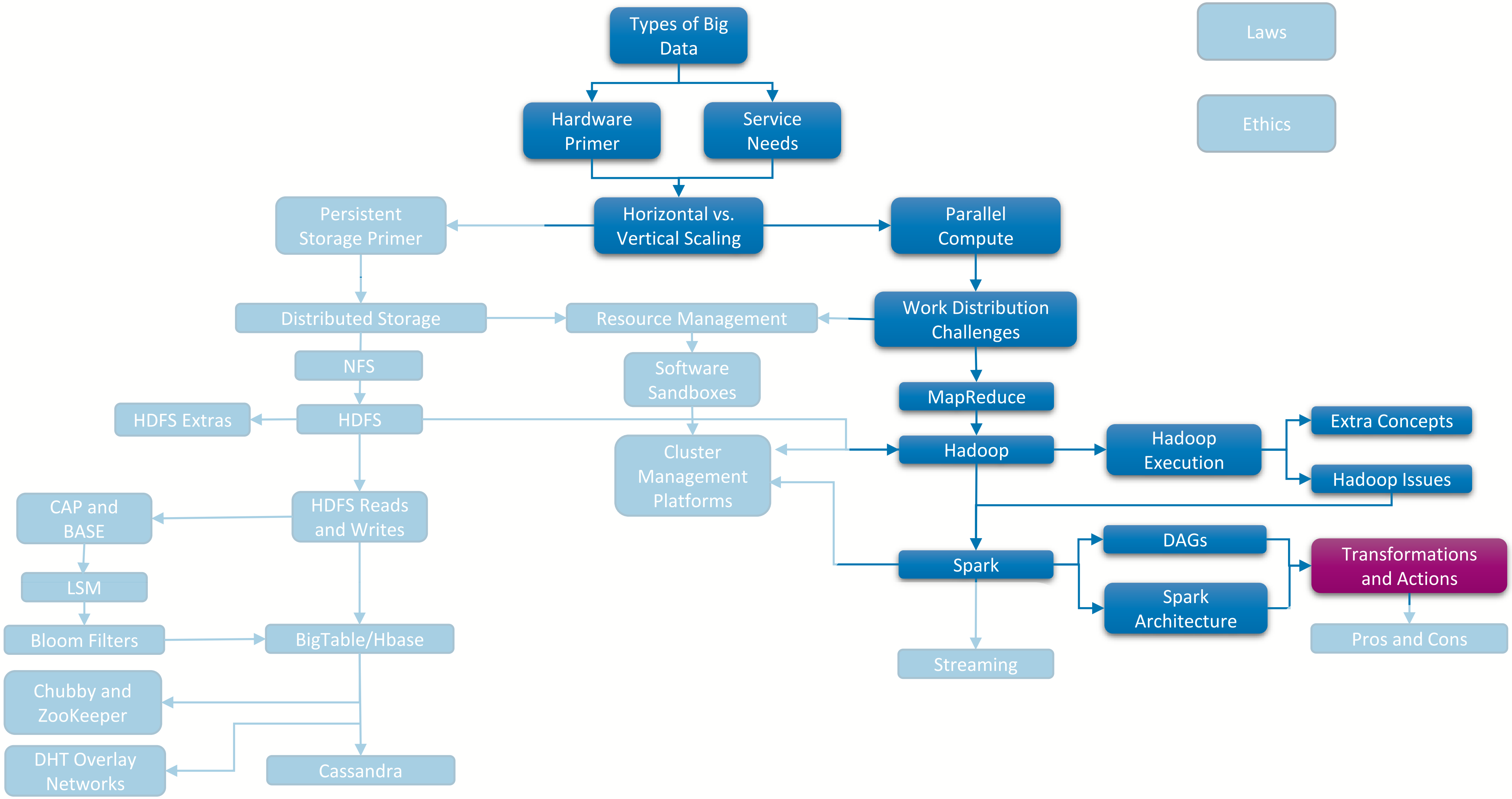
# **W4-03:** Spark Advanced Transformations

COMPSCI4064 (H) and COMPSCI5088 (M)

Dr. Richard McCreadie

# Where Are We?

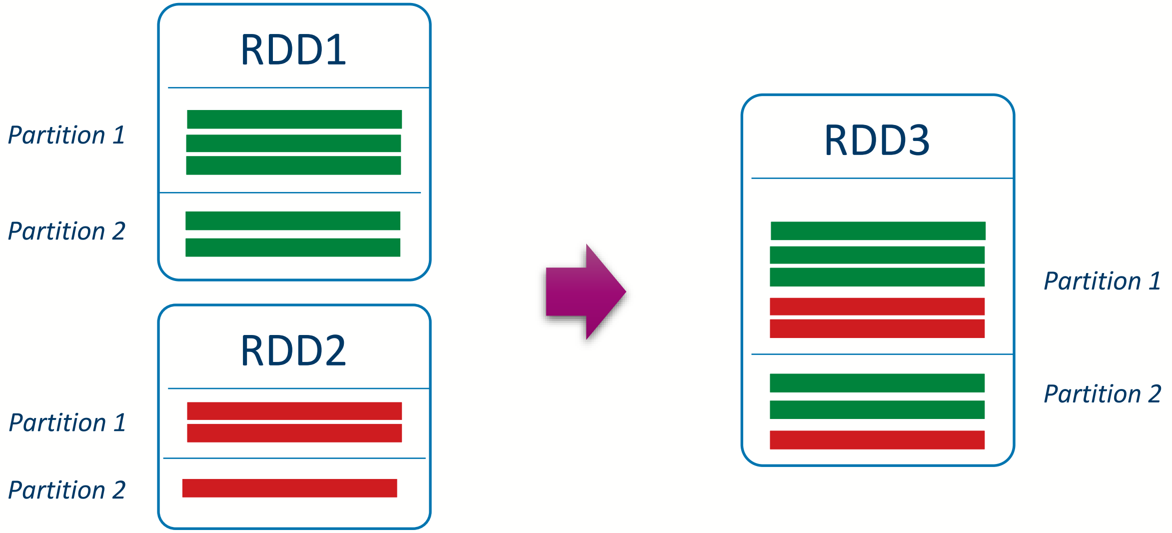

Types of Big Data
Hardware Primer
Service Needs
Laws
Ethics
Persistent Storage Primer
Horizontal vs. Vertical Scaling
Parallel Compute
Distributed Storage
Resource Management
Work Distribution Challenges
NFS
Software Sandboxes
MapReduce
HDFS Extras
HDFS
Extra Concepts
Cluster Management Platforms
Hadoop
Hadoop Execution
Hadoop Issues
CAP and BASE
HDFS Reads and Writes
DAGs
Transformations and Actions
Spark
LSM
Spark Architecture
Bloom Filters
BigTable/Hbase
Pros and Cons
Streaming
Chubby and ZooKeeper
DHT Overlay Networks
Cassandra

# Set Transformations

Working with duplicate records

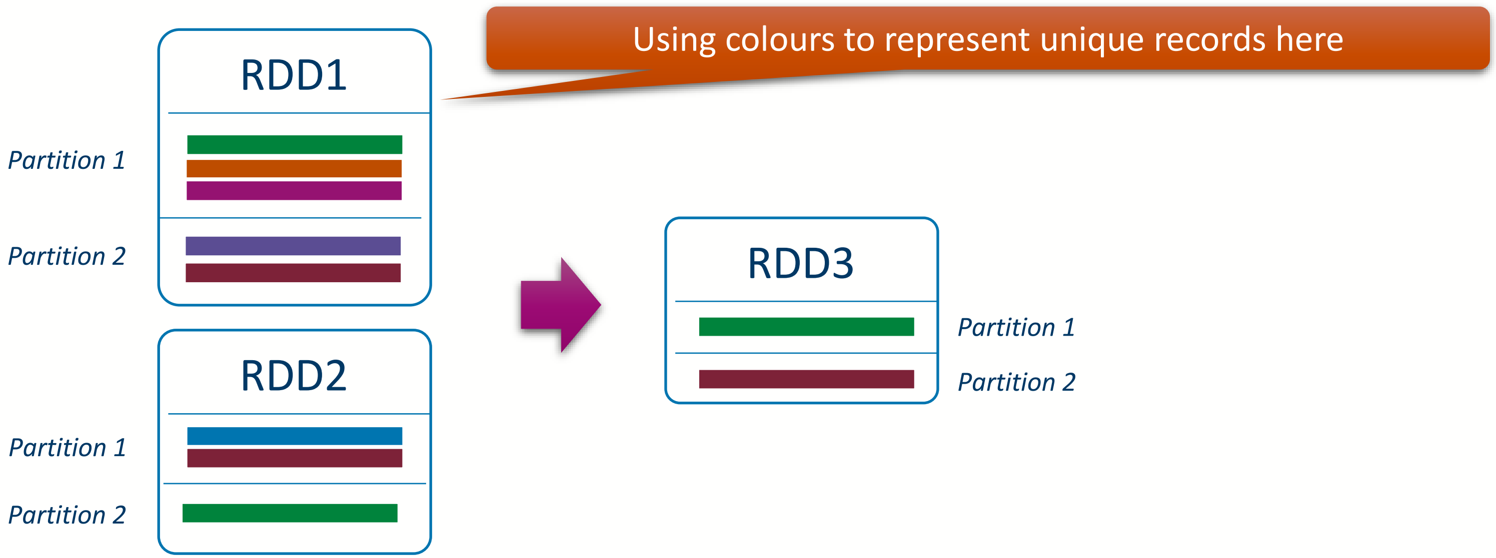
# **union**(*otherDataset*)

- Description: Return a new dataset that contains the union of the elements in the source dataset and the argument.
- Function: Built-in
  - Transformation Form: <RDD1<record1>,RDD2<record1>> ➡ RDD3<record1>
- Application: Note that as this is a union, if two records are identical, then only one will be kept in the output RDD



# intersection(*otherDataset*)

- Description: Return a new RDD that contains the intersection of elements in the source dataset and the argument.
- Function: Built-in
  - Transformation Form: <RDD1<record1>,RDD2<record1>> ➡ RDD3<record1>
- Application: Only records that are identical in both input RDDs will be kept.

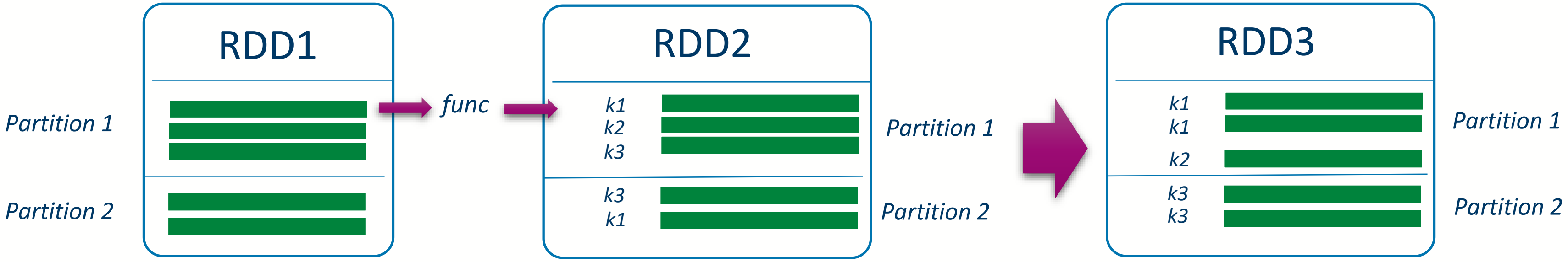


# Key Transformations

Working with <Key,Value> Pairs

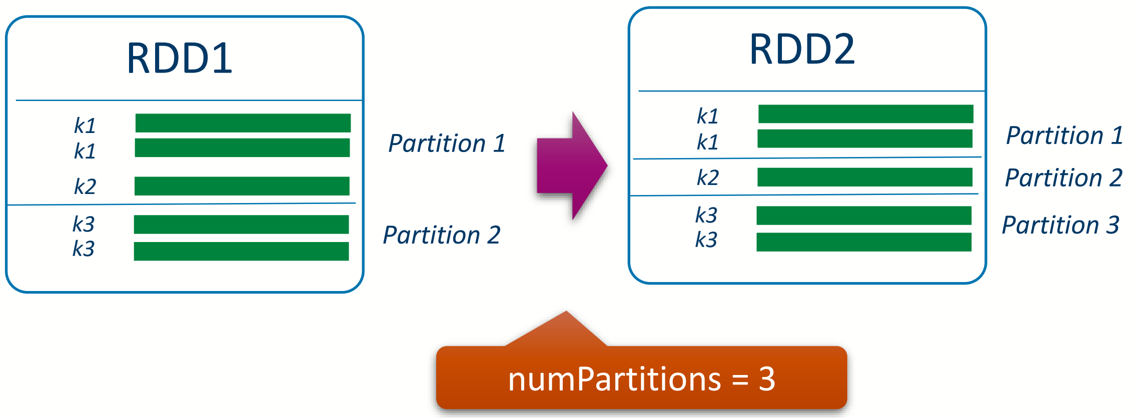
# groupByKey(func)

- Description: Converts an RDD<V> to an RDD<K,V> using a function func, which for a record generates a key, then groups the records by key into partitions.
- Function: record1 ➡ key   MapFunction
- Application: For each record in the RDD, the function will be used to generate a key, then that record will be added to a tuple with that key. When grouping one partition may contain multiple keys.



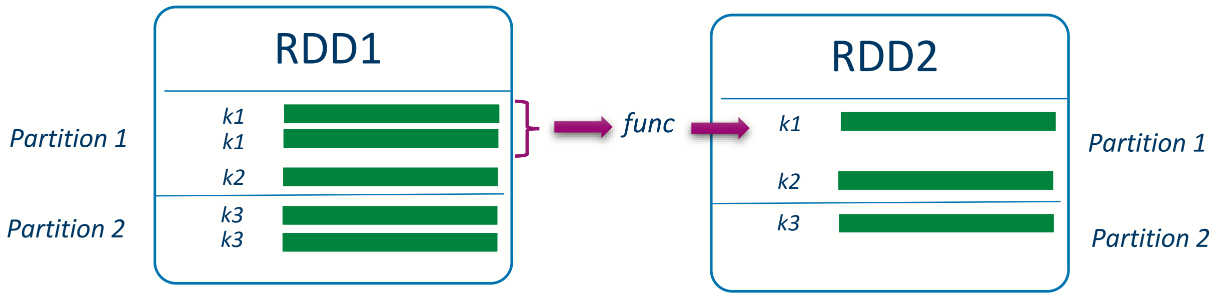
# groupByKey([*numPartitions*])

- Description: When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs.
- Function: Built-In
  - Transformation Form: RDD<key,record1> ➡ RDD<key,List<record1>>
- Application: You can also use groupByKey to re-partition your RDDs



# **reduceByKey**(*func*, [*numPartitions*])

- Description: When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function func, which must be of type (V,V) => V. Like in groupByKey, the number of reduce tasks is configurable through an optional second argument.
- Function: record1,record1 ➡ record1 — ReduceFunction
- Application: Uses a Reduce function to perform the reduction.



# **aggregateByKey**(*zeroValue*)(*seqOp*, *combOp*, [*numPartitions*])

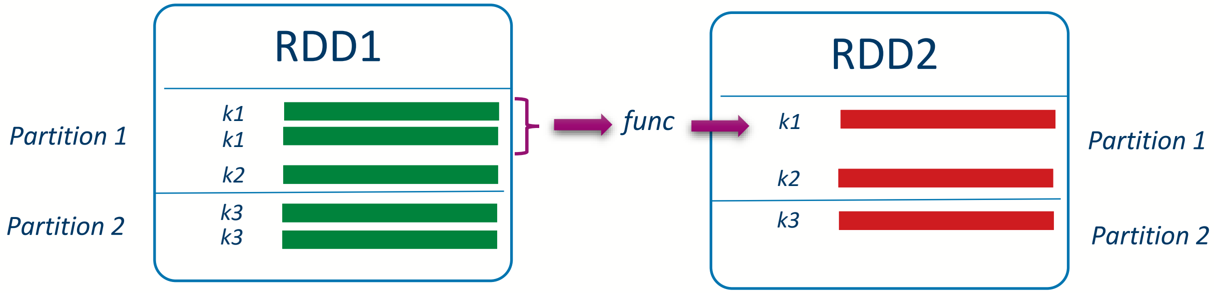
- Description: When called on a dataset of (K, V) pairs, returns a dataset of (K, U) pairs where the values for each key are aggregated using the given combine functions and a neutral "zero" value. Allows an aggregated value type that is different than the input value type, while avoiding unnecessary allocations. Like in groupByKey, the number of reduce tasks is configurable through an optional second argument.

Record2 is an accumulator

- Seq Function: record1,record2 ➡ record2

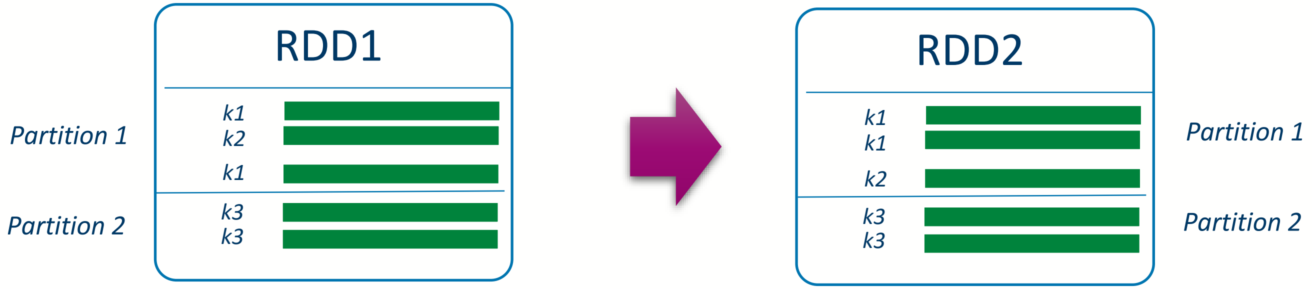
ReduceFunction

- Comb Function: record2,record2 ➡ record2
- Application: The sequence function is applied to each partition, the combination function is applied across the output of each partition



# sortByKey([*ascending*], [*numPartitions*])

- Description: When called on a dataset of (K, V) pairs where K implements Ordered, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean ascending argument.
- Function: Sort
- Application: Note that the keys need to have an ordering

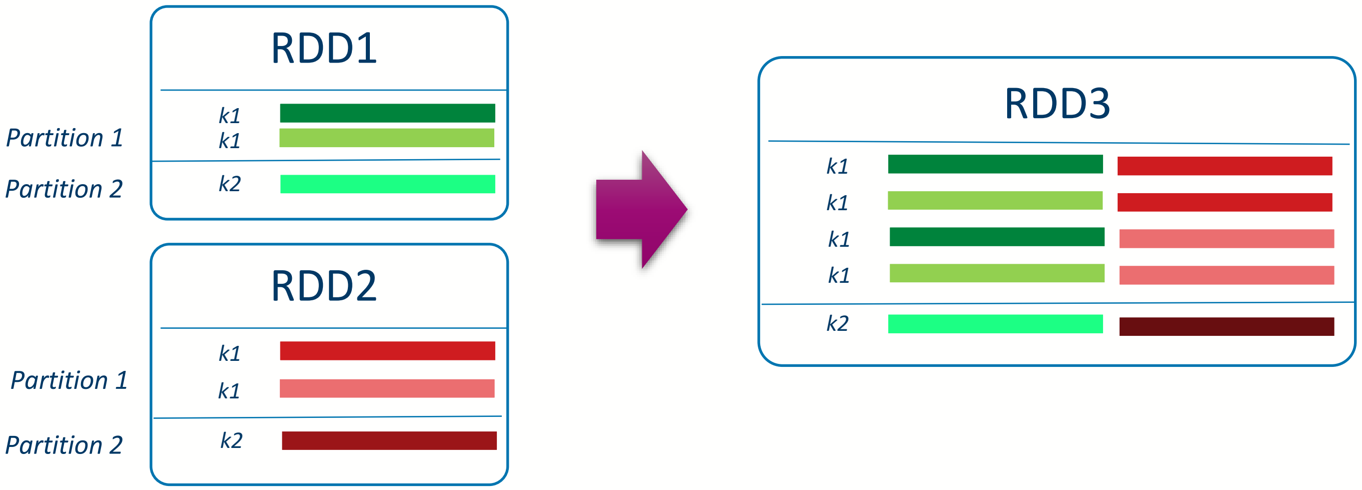


# Set Key Transformations

Expensive Set Transforms

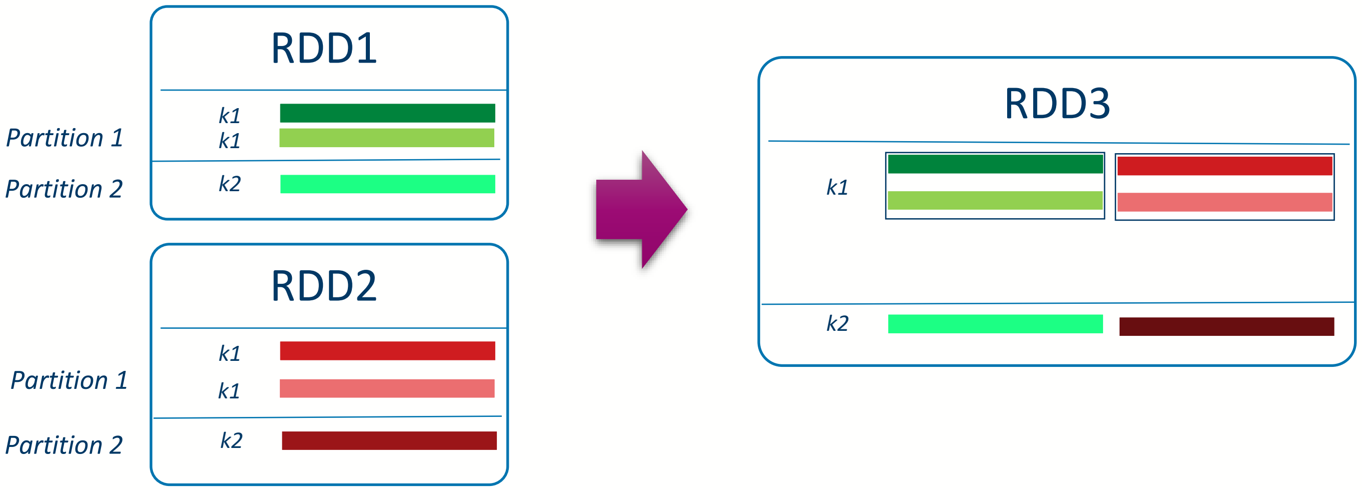
# join(*otherDataset*, [*numPartitions*])

- Description: When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are supported through leftOuterJoin, rightOuterJoin, and fullOuterJoin.
- Function: None
  - Transformation Form: <RDD1<key,record1>,RDD2<key,record2>> ➡ RDD3<key,<record1,record2>>
- Application: ‘Table’ join, very expensive



# **cogroup**(*otherDataset*, [*numPartitions*])

- Description: When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (Iterable<V>, Iterable<W>)) tuples. This operation is also called groupWith.
- Function: None
  - Transformation Form: <RDD1<key,record1>,RDD2<key,record2>> ➡ RDD3<key,List<record1>,List<record2>>

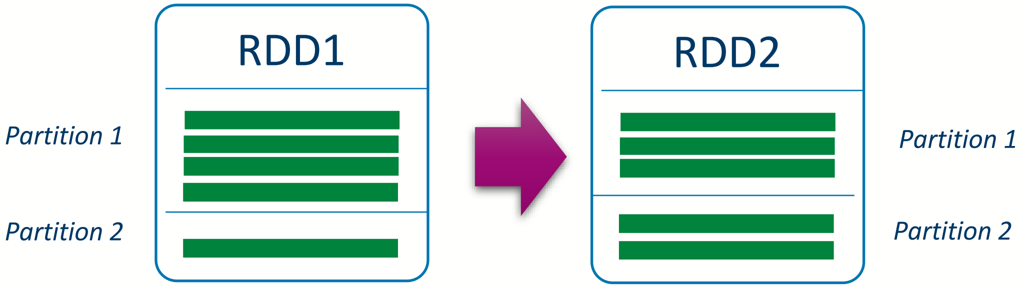


# Re-Partitioning

Changing the partitions

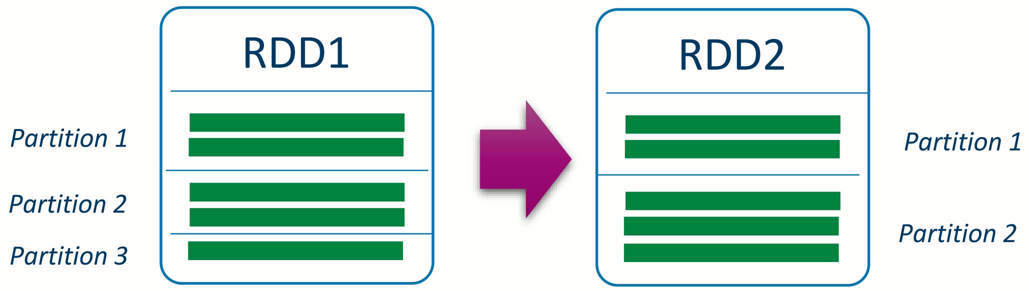
# **repartition**(*numPartitions*)

- Description: Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.
- Function: None
- Application: Re-balancing your data across partitions



# coalesce(*numPartitions*)

- Description: Decrease the number of partitions in the RDD to numPartitions. Useful for running operations more efficiently after filtering down a large dataset.
- Function: None
- Application: This merges partitions together



Course Coordinator
**Dr. Richard McCreadie**

**Email:** richard.mccreadie@glasgow.ac.uk
**Room:** SAWB 304

**#UofGWorldChangers**
**@UofGlasgow**